

Subgraphs & Workflow Orchestrators

Session Notes — AI Engineering Course

Topics: Subgraph · Graph vs Subgraph · 6 Workflow Patterns · Send API · Orchestrator-Worker · Inventory Example

❖ 1. What is a Graph?

A **graph** is a collection of **nodes** and **edges**.

→ **Node** — a Python function. Performs one specific unit of work.

→ **Edge** — the connectivity between nodes. Defines control flow.

→ **Graph** = nodes connected by edges = a workflow.

In LangGraph, we use the graph to implement the **Agentic flow**.

Example: $N1 \rightarrow N2 \rightarrow N3$, each node is a different function, edges connect them in sequence.

❖ 2. What is a Subgraph?

A **subgraph** is a compiled graph that is **ingested as a node** inside a parent graph.

→ Graph 1 (has its own nodes and edges)

→ Graph 2 = Parent Graph

→ Graph 1 is added as a node inside Graph 2 → Graph 1 becomes a **subgraph**

The name is self-explanatory: a subgraph is a graph that is a **part of a parent graph**.

Key rules for subgraph integration:

Scenario	How to integrate
Subgraph shares state schema with parent	Add directly: <code>graph.add_node('name', subgraph)</code>
Subgraph has a different state schema	Wrap in a function that transforms state in/out; add that function as a node

State sharing: State is shared between the parent graph and the subgraph automatically when schemas match.

Compilation: Subgraphs are compiled separately first, then passed as a node into the parent graph.

❖ 3. Real Example — Inventory Management System

This example demonstrates a subgraph inside a parent graph.

State Schema (OrderState)

Field	Description
product	Name of the product being ordered

quantity	How many units requested
inventory_status	Available or Not Available (set by subgraph)
total_price	Calculated price (set by subgraph)
final_response	Human-readable response (set by parent)

Subgraph — what it does:

Subgraph nodes:

→ **check_inventory**: reads product from state, checks product catalog (dict/DB), compares quantity to stock.

If stock $\geq$ quantity → inventory_status = 'Available'

Else → inventory_status = 'Not Available'

→ **calculate_price**: reads inventory_status. If not available → total_price = 0.

Else → unit_price = product_catalog[product]['unit_price']

total_price = quantity × unit_price

Subgraph edges: START → check_inventory → calculate_price → END

Parent Graph — what it does:

→ validate_order node — validates the incoming order

→ order_processing node — this is where the subgraph is ingested

→ final_response node — generates the human-readable output

The subgraph runs inside order_processing. Because both graphs share OrderState, state values pass between them automatically.

❖ 4. Send API

Send sends requests to nodes or subgraphs **in parallel**.

→ Previously seen APIs: MessagesState, different edge types, interrupt API.

→ Send API is new. It dispatches requests to multiple nodes/subgraphs simultaneously.

→ Each dispatch runs in its own thread — parallel execution.

Key use: **Orchestrator-Worker pattern** and **fan-out / fan-in**.

	Regular Edge	Send API
Defined at	Compile time (graph.compile())	Runtime (inside a node function)
Branches	Fixed — same every run	Dynamic — decided during execution
Parallelism	No	Yes — runs multiple threads simultaneously
Connection type	Normal or conditional edge	Always via conditional edge

❖ 5. 6 Workflow Orchestrators in LangGraph

These are the main workflow patterns used to design Agentic systems in LangGraph. Knowing these lets you read and write any graph-based AI system.

5.1 Sequential Workflow

Data flows through nodes in a **fixed sequence**.

$N1 \rightarrow N2 \rightarrow N3 \rightarrow N4$

Key mechanism: Normal edges only.

Use when: Steps always happen in the same order with no branching.

5.2 Parallelization — Fan-out / Fan-in

Fan-out: One node passes output to multiple nodes simultaneously.

Node $\rightarrow$ [Node A, Node B, Node C] (parallel execution)

Fan-in: Multiple nodes pass output to one single node.

[Node A, Node B, Node C] $\rightarrow$ Combiner Node

Key mechanism: Send API + Annotated[list, operator.add] reducer.

Reducer: Without it, parallel writes overwrite each other. operator.add appends instead.

5.3 Conditional Routing (Routing)

Based on state, the graph **dynamically selects one branch** to execute.

Node $\rightarrow$ [Branch A OR Branch B] (only one runs)

Key mechanism: Conditional edge + router function.

Also called: Conditional edge, dynamic branch.

Example: tools_condition — routes to ToolNode if tool_calls exists, else END.

5.4 Orchestrator-Worker

Fan-in: Use `Annotated[list, operator.add]` to collect all worker outputs.

Limit: Set a max attempt count (e.g. 3) to prevent infinite loops.

The model controls which tool runs — not the graph edges.

Pattern	Flow type	Key mechanism	Dynamic?
Sequential	Fixed sequence	Normal edges	No
Fan-out / Fan-in	Parallel branches	Send + operator.add reducer	Yes
Routing	One conditional branch	Conditional edge + router	Yes

Orchestrator-Worker	Dynamic parallel fan-out	Send via conditional edge	Yes
Evaluator-Optimizer	Feedback loop	Conditional edge + retry limit	Yes
ReAct Loop	Model-driven tool calls	bind_tools + tools_condition	Yes

❖ 7. What's Coming Next

Session	Topic
Next class	Multi-Agentic Flow
After that	MCP (Model Context Protocol) + Agent-to-Agent (A2A)
Class 49	Guardrails
Class 50–51	Evaluation (DeepEval framework)
After syllabus	End-to-end Projects + Cloud Deployment (Azure)

❖ 8. Key Reminders

■ Subgraph state — different schema = wrapper function required
If the subgraph state schema differs from the parent, wrap in a function that transforms state in and out. Don't add the compiled graph directly.

■ Send API uses a conditional edge — not a normal edge
The worker is connected to the orchestrator via a conditional edge.
The Send dispatches through that conditional edge at runtime.

■ Subgraph is compiled first, then passed as a node to the parent
`order_subgraph = order_subgraph_builder.compile()`
`parent.add_node('order_processing', order_subgraph)`

■ Shared checkpointer across Kubernetes pods = fault-tolerant HITL
If one pod fails mid-execution, another pod with the same thread_id can load the last checkpoint and resume. This is the production HITL pattern.

❖ 9. Challenge 5 — Orchestrator-Worker with Send API

Task: Build an orchestrator-worker graph.

- User provides a subject (e.g. 'AI Engineering')
- Orchestrator node generates 3 subtopics
- Send dispatches each subtopic to a research worker in parallel
- Each worker writes a short research paragraph
- Combiner node merges all 3 into one final report

Constraints:

- from langgraph.types import Send
- State reducer: Annotated[list, operator.add]
- Orchestrator and worker MUST have different state schemas

Reference: <https://langchain-ai.github.io/langgraph/how-tos/map-reduce/>